

IMPLEMENTACIÓN Y COMPARACIÓN DE ALGORITMOS DE BÚSQUEDA

Nombre: Diego Aido

Asignatura: Programación Avanzada

1. INTRODUCCIÓN

El presente informe documenta el diseño, implementación y rendimiento de cuatro algoritmos fundamentales de la teoría de grafos y la inteligencia artificial: Búsqueda en Anchura (BFS), Búsqueda en Profundidad (DFS), Búsqueda de Costo Uniforme (UCS) y Búsqueda A*. Estos algoritmos fueron evaluados en dos entornos diferenciados: un grafo explícito ponderado y un laberinto bidimensional representado mediante matrices.

2. DECISIONES DE DISEÑO Y ARQUITECTURA

Con el fin de construir un software modular, mantenible y escalable, se optó por una arquitectura desacoplada dividida en módulos independientes:

A. Modularidad y Separación de Entornos

- `main.py`: Funciona como el punto de entrada. Proporciona una interfaz limpia por consola para que el usuario navegue entre los escenarios.
- `grafo_busqueda.py`: Encapsula la lógica de la parte 1. Utiliza diccionarios nativos de Python para representar la lista de adyacencia del grafo.
- `laberinto_busqueda.py`: Encapsula la lógica de la parte 2. Maneja los desplazamientos espaciales mediante coordenadas cartesianas (f, c) y restringe los movimientos a las cuatro direcciones cardinales.

B. Interfaz y Visualización Textual

Se descartaron librerías gráficas pesadas (como `matplotlib` o `networkx`) en favor de una visualización puramente textual en consola. Esto garantiza:

1. Portabilidad absoluta: El código se ejecuta de manera idéntica en cualquier sistema operativo o terminal sin dependencias externas.

C. Modelado de Heurísticas

- En Grafos: Se utilizó un diccionario estático que simula la distancia en línea recta hacia el nodo objetivo.
- En Laberintos: Se implementó la Distancia de Manhattan, definida analíticamente como:
$$\text{Manhattan} = |x_1 - x_2| + |y_1 - y_2|$$

Esta elección es la matemáticamente correcta para laberintos donde no se permiten movimientos diagonales, siendo una heurística admisible (nunca sobreestima el costo real) y consistente.

3. COMPARACIÓN Y ANÁLISIS DE LOS ALGORITMOS

A partir de las pruebas ejecutadas en ambos entornos, se derivan las siguientes observaciones de rendimiento:

A. BFS

- Mecanismo: Utiliza una cola FIFO. Explora el entorno por niveles (ondas concéntricas).
- Resultados: En el laberinto, garantiza encontrar el camino con el menor número de pasos. Sin embargo, su coste de memoria es elevado debido a que expande todos los nodos de un nivel antes de pasar al siguiente. En el grafo, al no considerar los pesos de las aristas, puede entregar rutas subóptimas en términos de costo acumulado.

B. DFS

- Mecanismo: Utiliza una pila LIFO. Explora una rama por completo antes de retroceder.
- Resultados: Es el algoritmo más ineficiente para encontrar rutas mínimas. En el laberinto, genera caminos sumamente erráticos ("serpenteantes") y largos, ya que no tiene noción de cercanía al objetivo. Su única ventaja es un menor consumo de memoria en estructuras sumamente ramificadas si el objetivo es profundo.

C. UCS

- Mecanismo: Utiliza una cola de prioridad basada exclusivamente en el costo acumulado $g(n)$.
- Resultados: En el grafo con pesos variables, es indispensable, ya que garantiza la ruta óptima sumando los costos reales de las aristas. En el laberinto (donde cada paso tiene un costo uniforme de 1), su comportamiento y el orden de exploración de celdas resultaron idénticos a los de BFS.

D. A*

- Mecanismo: Utiliza una cola de prioridad basada en la función de evaluación $f(n) = g(n) + h(n)$.
- Resultados: Demostró ser el algoritmo más eficiente en el laberinto. Al combinar el costo real con la distancia de Manhattan, el algoritmo exhibe un comportamiento "dirigido" u orientado hacia la meta. Reduce drásticamente el número de celdas marcadas como visitadas en comparación con BFS y UCS, ahorrando tiempo de cómputo y memoria.

4. CUADRO COMPARATIVO RESUMEN

Criterio	BFS	DFS	UCS	A*

Estructura Principal	Cola (FIFO)	Pila (LIFO)	Cola de Prioridad	Cola de Prioridad
Completo (¿Halla solución?)	Sí (si el factor de ramificación es finito)	Sí (en espacios finitos)	Sí	Sí
Óptimo	Solo si los costos son idénticos	No	Sí (siempre que $c \geq 0$)	Sí (con heurística admisible)
Uso de Memoria	Alto	Bajo (en comparación con BFS)	Alto	Moderado/Bajo (Filtra nodos inútiles)
Uso Ideal	Árboles sin pesos, rutas cortas en pasos.	Recorridos completos de estructuras.	Grafos con pesos donde se desconoce la geometría del espacio.	Laberintos, mapas de videojuegos y sistemas de navegación GPS.

5. PRUEBAS DE EJECUCIÓN

a) BFS:

```

1. Ejecutar en Grafos / Árboles (Parte 1)
2. Ejecutar en Laberinto de Matrices (Parte 2)
Seleccione una opción: 1

--- BÚSQUEDA EN GRAFOS (TEXTUAL) ---
Nodos disponibles: ['A', 'B', 'C', 'D', 'E', 'F', 'G']
Nodo inicial (Ej: A): a
Nodo objetivo (Ej: G): g

Seleccione algoritmo:
1. BFS
2. DFS
3. UCS
4. A*
Opción: 1

=====
VISUALIZACIÓN DEL RECORRIDO
=====
Orden de exploración: A -> B -> C -> D -> E -> F -> G
Camino encontrado : A -> B -> D -> G
-----

Estructura del Grafo (Aristas y costos):
Nodo A [Camino] -> B(peso:1), C(peso:4)
Nodo B [Camino] -> A(peso:1), D(peso:5), E(peso:2)
Nodo C [Visitado] -> A(peso:4), F(peso:3)
Nodo D [Camino] -> B(peso:5), G(peso:2)
Nodo E [Visitado] -> B(peso:2), G(peso:6)
Nodo F [Visitado] -> C(peso:3), G(peso:1)
Nodo G [Camino] -> D(peso:2), E(peso:6), F(peso:1)

Costo total del camino: N/A

```

b) A*:

```

1. Ejecutar en Grafos / Árboles (Parte 1)
2. Ejecutar en Laberinto de Matrices (Parte 2)
Seleccione una opción: 1

--- BÚSQUEDA EN GRAFOS (TEXTUAL) ---
Nodos disponibles: ['A', 'B', 'C', 'D', 'E', 'F', 'G']
Nodo inicial (Ej: A): a
Nodo objetivo (Ej: G): g

Seleccione algoritmo:
1. BFS
2. DFS
3. UCS
4. A*
Opción: 4

=====
VISUALIZACIÓN DEL RECORRIDO
=====
Orden de exploración: A -> B -> E -> C -> D -> F -> G
Camino encontrado : A -> B -> D -> G
-----

Estructura del Grafo (Aristas y costos):
Nodo A [Camino] -> B(peso:1), C(peso:4)
Nodo B [Camino] -> A(peso:1), D(peso:5), E(peso:2)
Nodo C [Visitado] -> A(peso:4), F(peso:3)
Nodo D [Camino] -> B(peso:5), G(peso:2)
Nodo E [Visitado] -> B(peso:2), G(peso:6)
Nodo F [Visitado] -> C(peso:3), G(peso:1)
Nodo G [Camino] -> D(peso:2), E(peso:6), F(peso:1)

Costo total del camino: 8

```

c) DFS:

```

Dimensiones del laberinto: 7x7
Fila inicial (0-6): 0
Columna inicial (0-6): 0
Fila objetivo (0-6): 6
Columna objetivo (0-6): 6

Seleccione algoritmo:
1. BFS
2. DFS
3. UCS
4. A*
Opción: 2

=====
MAPA DEL LABERINTO EN CONSOLA
=====
Leyenda: I=Inicio | O=Objetivo | █=Pared | *=Ruta | v=Visitado | -=Vacío

  0 1 2 3 4 5 6
0 I * * * * *
1 - █ █ █ █ *
2 - - - █ *
3 - █ █ █ * █
4 - █ █ █ █ *
5 █ █ █ █ █
6 - - - - - O

=====
Orden de exploración de coordenadas:
[(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (0, 6), (1, 6), (2, 6), (2, 5), (3, 5), (4, 5),
(4, 6), (5, 6), (6, 6)]

Camino final (coordenadas): [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (0, 6), (1, 6), (2,
6), (2, 5), (3, 5), (4, 5), (4, 6), (5, 6), (6, 6)]
Costo total (Número de pasos): N/A

```

d) UCS:

```

Fila inicial (0-6): 0
Columna inicial (0-6): 0
Fila objetivo (0-6): 6
Columna objetivo (0-6): 6

Seleccione algoritmo:
1. BFS
2. DFS
3. UCS
4. A*
Opción: 3

=====
MAPA DEL LABERINTO EN CONSOLA
=====
Leyenda: I=Inicio | O=Objetivo | █=Pared | *=Ruta | v=Visitado | -=Vacío

  0 1 2 3 4 5 6
0 I v v v v v v
1 * █ █ █ █ v
2 * * * █ v v
3 v █ █ █ █ █
4 v v █ █ █ █
5 █ v █ █ █ █
6 v v v v v v O

=====
Orden de exploración de coordenadas:
[(0, 0), (0, 1), (1, 0), (0, 2), (2, 0), (0, 3), (2, 1), (3, 0), (0, 4), (2, 2), (4, 0), (0, 5),
(2, 3), (4, 1), (0, 6), (3, 3), (5, 1), (1, 6), (4, 3), (6, 1), (2, 6), (4, 4), (6, 0), (6, 2),
2, 5), (4, 5), (6, 3), (3, 5), (4, 6), (6, 4), (5, 6), (6, 5), (6, 6)]

Camino final (coordenadas): [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (3, 3), (4, 3), (4,
4), (4, 5), (4, 6), (5, 6), (6, 6)]
Costo total (Número de pasos): 12

```

6. CONCLUSIONES

La realización de este proyecto evidencia que no existe un algoritmo de búsqueda absoluto para todos los problemas. Mientras que las búsquedas ciegas o no informadas (BFS y DFS) son útiles cuando no poseemos información del entorno más allá de su conectividad, la inclusión de una función heurística en A* transforma por completo la eficiencia del proceso.

En entornos espaciales como los laberintos matriciales, A* destaca notablemente al evitar la exploración ciega de pasillos que se alejan de la meta, consolidándose como la solución óptima para sistemas de navegación y toma de decisiones en Inteligencia Artificial.